Incremental Evolution of Complex General Behavior

Faustino Gomez and **Risto Miikkulainen**
Department of Computer Sciences
The University of Texas at Austin, Austin, TX 78712
inaki,risto@cs.utexas.edu

Technical Report AI96-248
June 1996

Abstract

Several researchers have demonstrated how complex behavior can be learned through neuro-evolution (i.e. evolving neural networks with genetic algorithms). However, complex general behavior such as evading predators or avoiding obstacles, which is not tied to specific environments, turns out to be very difficult to evolve. Often the system discovers mechanical strategies (such as moving back and forth) that help the agent cope, but are not very effective, do not appear believable and would not generalize to new environments. The problem is that a general strategy is too difficult for the evolution system to discover directly. This paper proposes an approach where such complex general behavior is learned incrementally, by starting with simpler behavior and gradually making the task more challenging and general. The task transitions are implemented through successive stages of delta-coding (i.e. evolving modifications), which allows even converged populations to adapt to the new task. The method is tested in the stochastic, dynamic task of enemy avoidance, and compared with direct evolution. The incremental approach evolves more effective and more general behavior, and should also scale up to harder tasks.

1 Introduction

Neuro-Evolution(NE), the training of Artificial Neural Networks with Genetic Algorithms, has proven an attractive alternative to standard learning methods in reinforcement environments (Colombetti and Dorigo 1992; Nolfi et al. 1990; Nolfi and Parisi 1995; Yamauchi and Beer 1994). In NE, strings of values (chromosomes) representing neural network parameters (e.g. connection weights, thresholds, and connectivity) are recombined based on the principle of natural selection in order to find an optimal network for a given problem. Many NE systems consist of a mobile agent in a static environment where the agent must learn a policy that minimizes the travel distance between certain desirable states (Cliff et al. 1992; Law and Miikkulainen 1994; Nolfi and Parisi 1994). These "goal states" are defined spatially by unique coordinates so that it is easy to quantitatively assess an individual's competence by comparing the known optimal path to an attempted path. As tasks require increasingly complex behavior, this type of metric becomes difficult to define. Wall-following, obstacle-avoidance, and predator-prey scenarios are examples of such tasks because they describe a more general, abstract behavior not tied to specific spatial locations.

A critical problem in trying to evolve complex behaviors is that the strategies that emerge are often "mechanical." These strategies yield reasonable performance in terms of maximizing a fitness measure, but do not exhibit the kind of intelligent responsiveness to the environment that is required to ultimately accomplish the objective. Seeming competence in the task is often only a side-effect of the "mechanical" strategy; the agent does not learn the general task, but, instead, learns a relatively primitive, environment-specific behavior. For example, a prey being evolved to evade a predator may exhibit a simple strategy of moving back and forth or in a circle through the environment. Because this behavior is easy to evolve and guarantees better fitness than simply moving randomly, the population can easily be lured into a region of the solution space that manifests this strategy. Once in this pathological region, it is unlikely that a good solution will be found since the non-mechanical, general behavior required to accomplish this task can be very different from the mechanical behavior.

This paper demonstrates a method for evolving the complex general behavior incrementally. Instead of evaluating a population on the same task throughout the course of evolution, it is first evaluated on a relatively easy task and then on increasingly harder tasks, in an effort to build towards the goal behavior (Elman 1991). There are two techniques that allow us to do this: (1) ESP makes it possible to evolve neural networks that have memory, and (2) Delta-Coding makes it possible to adapt to each new task. The results of applying this method will show that it is possible to achieve complex general behavior with evolutionary neural nets.

2 Enemy Avoidance

Enemy Avoidance, the task used in this paper to demonstrate incremental evolution, is a special case of a class of problems known as Pursuit and Evasion. Pursuit and Evasion contests consist of an environment containing a minimum of two entities: a predator and a prey. The predator moves through the environment trying to capture the prey while the prey attempts to avoid capture by fleeing from the predator. Scenarios of this kind are a particularly interesting arena for the study of adaptive behavior because they are ubiquitous in natural ecosystems and offer a relatively simple objective that requires complex sensory-motor coordination with respect to both the environment and another moving entity (Miller and Cliff 1994). To accomplish the Enemy Avoidance task, an Agent, moving through a simulated environment, must exhibit a behavior which prevents it from coming into contact with a number of malevolent enemies for as long as possible. Enemy Avoidance is simple in both description and implementation, yet demands a level of behavioral sophistication that is susceptible to the emergence of "mechanical" strategies.

2.1 Environment

The environment consists of a square spatially and temporally discrete grid-world wherein two types of objects reside: the *Agent* and any number of *enemies* (figure 1a). Each object occupies a single grid space and can move in one of four directions {N,S,E,W} at each time-step. An enemy is considered to have come into contact with the Agent when both Agent and enemy occupy the same grid-cell. The enemies move probabilistically with a tendency to pursue the Agent that grows stronger as they get closer to the Agent (see Appendix A for a definition of the enemy algorithm, due to (Lin 1992)). Each enemy moves at a speed that is set between 0 and 1. This value is the probability of an enemy taking an action each time-step. An enemy with a speed of 0.5 will do nothing 50% of the time. Note that if the environment were continuous, a speed of 0.5 would make the task quite easy because the enemy would always be moving at the same leisurely rate. In the discrete world of Enemy Avoidance, however, an enemy moving at a speed of 0.5 is really moving at the same speed as the Agent but only part of the time. Therefore, the Agent must learn to cope with dangerous periods in which the enemy happens to make consecutive moves (i.e. moves at the same speed as the enemy; one grid-cell per turn). For example, near the four corners of the small grid-world an enemy can easily catch the Agent with just four consecutive moves, and the Agent should stay away from those situations.

This environment provides a non-trivial task with a relatively high complexity. The Agent cannot simply learn a sense→action mapping that relies on predictable state transitions in order to successfully elude the enemies indefinitely. Instead, it must use a policy that encodes knowledge of worst-case enemy behavior and memory of its past sensory input. More formally, the environment can be described by the *sensory-state machine* (SSM)(Wilson 1991):

$$\{E(t+1)\} = f(E(t), A(t)), \tag{1}$$

where $E(t)$ and $A(t)$ are the Agent's sensory stimulus and action at time t, respectively, and f is the mapping that describes the environmental dynamics from the Agent's perspective. For a given sensory stimulus (sense vector) $E(t)$ and an action $A(t)$ there is a finite set of sensory stimuli $\{E(t+1)\}$ which *may* result. If $cardinality(\{E(t+1)\}) > 1$ for any pair $(E(t), A(t))$, then the environment is non-deterministic. When the sense vector $E(t)$ does not denote a unique global state, that is, the Agent cannot sense the entire environment with total accuracy, some form of memory is required to help the Agent predict where the

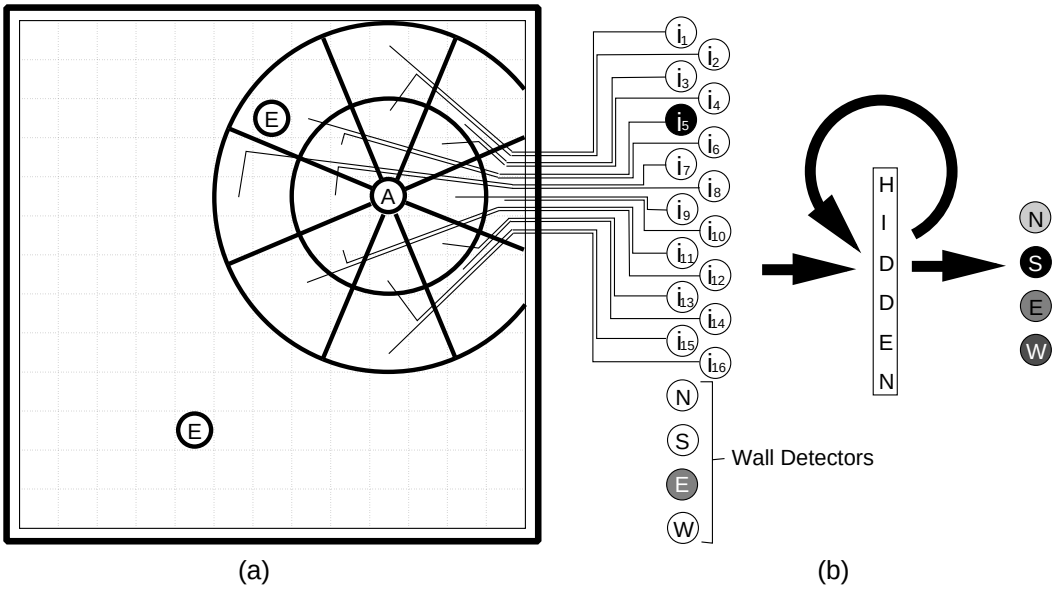


Figure 1: The Enemy Avoidance Environment and the Agent Network. (a) The 13×13 grid world occupied by the Agent and two enemies. The sectors and circles around the Agent represent its sensory array. There are 8 sectors divided into two levels of proximity. Each sensory area is represented by a node in the input of the neural network controlling the Agent, as shown in (b). An input unit is activated when an enemy enters the corresponding area. Each of the wall detector units is activated to a degree that is proportional to the Agent's distance to the wall in that direction, provided the wall is within the sensor range. In the situation shown here, the input unit I_5 is activated because an enemy is in the far NW area. The E wall-detector unit is also activated by a small amount because the east wall is just within sensor range. This input is fed into the hidden layer along with the hidden layer activation from the previous time-step. The new hidden layer activation is then propagated into the output layer. In this scenario, the Agent will move south because the south (S) output unit has the highest activation.

enemies are. In other words, the uncertainty about $E(t+1)$ can be reduced by keeping track of previous sense and action vectors:

$$\{E(t+1)\} = f_2(E(t), A(t), E(t-1), A(t-1)...E(t-k), A(t-k)), \tag{2}$$

If the environment is deterministic and contains a finite number of states, then there exists some k for which $cardinality(\{E(t+1)\}) = 1$ for all possible pairs $(E(t), A(t))$ and the Agent can thereby reliably predict $\{E(t+1)\}$. The Enemy Avoidance environment presented here is particularly difficult because there does not exist a k for which the non-determinism in (2) is eliminated. Due to the enemies' stochastic nature, the Agent can never learn to predict $E(t+1)$ exactly, although it can reduce the uncertainty with memory.

2.2 Agent Representation

The Agent is controlled by a recurrent two-layer neural network with sigmoid units (Figure 1b). Each layer is fully connected to its adjacent layer(s) and each unit in the hidden layer is connected to itself and to all other hidden units. Such recurrency affords the Agent temporal information which is necessary for performing the task.

As the Agent moves through the environment it can detect the presence of enemies within a specified sensor range (figure 1a). There is one input unit (I_i) assigned to each of the 16 areas in the sensory array. When an enemy enters an area covered by the sensory array, the unit corresponding to that area is set to `1`.

The units I_1 through I_{16} therefore afford a coarse encoding of relative enemy position. The radial nature of the sensory apparatus gives greater sensitivity to enemy movement at close range, where it is most crucial. Four more units are used to detect the walls in the N, S, E and W directions. As a wall comes within sensor range the corresponding unit is activated to a degree that is proportional to the wall's distance from the Agent. There is one output unit for each of the four possible actions. At each time step the Agent selects the action corresponding to the unit with the highest activation. This representation provides the Agent with sensory input that is both imprecise and of limited range. In order to perform well in Enemy Avoidance, an Agent must learn to associate sensory input with appropriate evasive actions and exploit its short-term memory (recurrent connections) to remember where it last saw an enemy if it moves out of sensor range.

3 Incremental Evolution

By attacking a difficult task such as Enemy Avoidance "head-on" from the outset of an evolution process, conventional evolutionary methods suffer from inefficient performance. The task is too complex to exert significant selective pressure on the population during the early stages of evolution. All of the individuals perform poorly and therefore the GA gets trapped in an unfruitful region of the solution space. If a population is first evolved on an *easier* version t' of the complex task t, it may be possible to discover a region of the network space from which the t is more accessible. If this is not the case, it may be possible to evolve a population on an even easier version t'' of t from which t' is more accessible, and so on. In this way, the ultimate task can be achieved by evolving incrementally on a sequence of tasks starting with a task that can be evolved directly.

In order to formulate a scheme for incremental evolution, it is necessary to make a distinction between the *evaluation-task* which is used to evaluate the Agent's fitness for reproduction, and the *goal-task* which the agent is evolved to perform (Nolfi and Parisi 1994) (*task* connotes both the environment and the behavioral objective). The goal-task can then be regarded as the culmination of a series of progressively more demanding evaluation-tasks: $\{t_1, t_2, t_3, \ldots, t_n\}$ where n is the total number of evaluation-tasks and t_n is the goal-task. This set of tasks is ordered so that t_i is easier than t_{i+1} for all i: $0 < i \leq n$. The number of tasks required (n) must be determined experimentally.

Each evaluation-task is derived by transforming a goal-task specification to one where the appropriate behavior is more readily "evolvable." A task transformation must preserve the underlying structure of the environment as well as the overall goal-task objective. The following two heuristics can be applied to derive effective evaluation-tasks: (1) Increase the "density" of the relevant experiences within a trial so that a network can be evaluated based on greater information in a shorter amount of time; and (2) Make the evaluation-task easier so that the acquisition of fundamental goal-task skills is more feasible. A combination of these two heuristics is used in the evolution of Enemy Avoidance. The evaluation-task environments are designed to temporally concentrate experience and to facilitate skill acquisition.

Once the set of evaluation-tasks has been derived, evolution of the goal-task behavior can proceed by evolving on each evaluation-task in succession. First, t_1 is evolved until a satisfactory level of performance is achieved. After t_1 is completed, t_2 is instantiated and evolution resumes. This process continues until the goal-task is completed.

Incremental evaluation attempts to give the problem of learning complex general behavior a competitive orientation that, in spirit, constitutes a cognitively plausible model of continual (or lifelong) learning (Elman 1991; Ring 1994; Thrun 1996). If, for instance, the goal-task is that of driving a Formula 1 race car at Grand Prix level, we can imagine a lattice of tasks arranged in order of increasing difficulty leading up to the goal-task. Tasks are arranged in a lattice because there may be many epistemic paths to accomplishing the goal-task. For this particular case, tasks might include: learning to drive a car safely, learning to drive fast, competing against novice drivers, competing in stock cars, and competing in rallies. One would not endeavor to drive a Formula-1 Car without first having gained competence in other forms of racing and, more importantly, learning how to drive a car in the first place. In the experiments that follow, this principle is exploited by having an Agent similarly compete against an environment that becomes more challenging in response to its growing expertise. Intuitively, the knowledge represented by a network evolved successfully

on a particular task should provide a favorable predisposition for learning a more difficult related task (Pratt 1993, 1994). This kind of knowledge transfer is achieved by sustaining genetic diversity in the population through a method called Delta-Coding.

4 Neuro-Evolution Method: *Enforced Sub-Populations (ESP)+Delta-Coding.*

The Neuro-Evolution method used is similar to Symbiotic, Adaptive Neuro-Evolution (SANE). SANE has been shown to be a very powerful reinforcement learning method for tasks with sparse reinforcement (Moriarty and Miikkulainen 1995, 1996a, 1996b). In order to apply this method to evolving complex general behavior, two extensions are necessary: Enforced Sub-Populations (ESP) to build recurrent networks, and Delta-Coding to assist in transferring to new tasks.

4.1 Enforced Sub-Populations

In ESP, as in SANE, the population consists of individual neurons instead of full networks. The best neurons are put together to form the agent network (its hidden layer). However, ESP allocates a separate population for each hidden unit in the network, and a neuron can only be recombined with members of its own sub-population. A segregation of the neuron population into dedicated sub-populations is crucial in evolving networks with recurrent connections. It also accelerates evolution in the kind of stochastic environment described above.

As in SANE, neuron fitness values are determined by the fitness of the networks in which they participate. Evolution in ESP proceeds as follows (figure 2):

1. Initialization. The number of hidden units n in the network architecture is specified and a sub-population (S) of chromosomes is initialized for each (S_1 through S_n). Each chromosome encodes the input, output, and recurrent connection weights of a neuron with real numbers chosen randomly from a uniformly distributed interval $[-x, x] \in \Re$ (where x is a fixed parameter).

2. Evaluation. One neuron is selected randomly from each of the n sub-populations to construct a complete network. This network is then submitted to a *trial* in which it is evaluated on the task and awarded a fitness score which is added to the *cumulative fitness* of each neuron in the network. This process continues until each neuron has participated in an average of 10 trials.

3. Recombination. For each sub-population, the average fitness of each neuron is calculated by dividing its cumulative fitness by the number of trials in which it participated. Neurons are then ranked within each sub-population by this average fitness. Each neuron in the top quartile is recombined with a higher-ranking neuron in the same sub-population (using 1-point crossover and mutation) to create the offspring to replace the lowest-ranking half of the sub-population.

4. Goto 2.

As in SANE, neurons compete on the basis of how well, on average, the networks in which they participate perform. A high average fitness means that the neuron contributes to forming successful networks and, consequently, suggests a good ability to cooperate with other neurons. In addition to the sub-population architecture, ESP deviates from SANE in that: 1. Recurrent connections are permitted, providing a mechanism for retaining knowledge of previous experience (states). 2. The network topology is *not* evolved (the network is fully connected). 3. Network weights are coded with real values instead of binary strings.

The idea of ESP is to allow for the evolution of recurrent connections at the neuron level by explicitly enforcing the specialization of neurons. In the advanced stages of the SANE evolution process, the neuron population clusters into "species" or groups of individuals that perform a specialized function in the target behavior (Moriarty and Miikkulainen 1995). SANE searches the space of possible neurons in a parallel fashion, sustaining diversity by randomly recombining members of different "species." In ESP, the groups

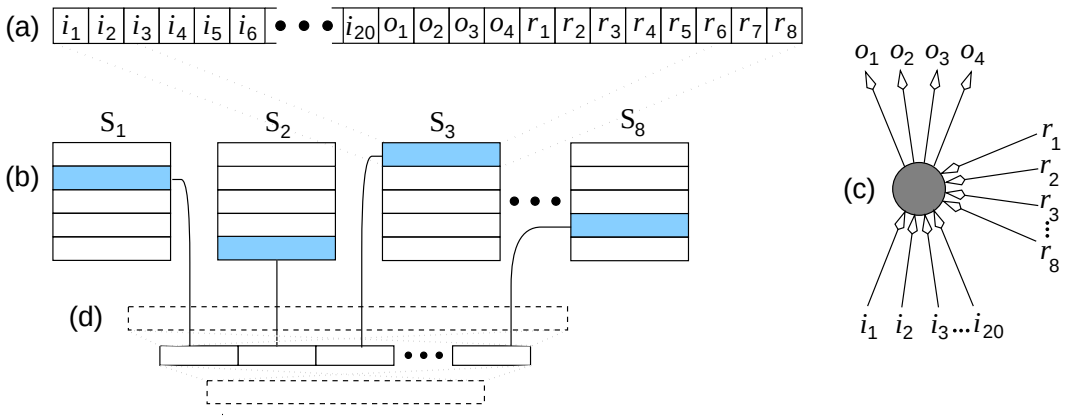


Figure 2: The Enforced Sub-Populations Method. Each element in the chromosome (a) encodes a connection weight for the neuron (c) with a single real number. The weights for connections *from* the input units are labeled i_1 through i_{20}, those for connections *to* the output units are labeled o_1 through o_4, and those *from* the hidden units are labeled r_1 through r_8. Neuron genotypes are segregated into Sub-Populations S_1 through S_8 (b) for a network of 8 hidden units (d). During evolution ESP forms networks by randomly selecting a neuron from each sub-population.

are already circumscribed by design, and their progressive specialization goes unencumbered by the recombination of "species" that may fulfill relatively orthogonal roles in the network mapping. Specialization is accelerated because the "species" do not have to organize themselves out of a single large population. Furthermore, because the networks formed by ESP always consist of a representative from each evolving specialization, a neuron is always evaluated on how well it performs its role in the context of all of the other players. In SANE, networks can contain redundant members of some developing clusters and omit members of others so that neurons are not allowed to assume a clear identity in the networks.

Specialization provides the basis for evolving effective recurrent networks. In the original SANE, a neuron's behavior in a recurrent network is critically dependent upon the neurons to which it is connected. Since SANE forms networks by randomly selecting neurons from a single population, a neuron cannot rely on being combined with the same neurons in any two trials. A neuron that behaves one way in one trial may behave very differently in another, and SANE cannot obtain accurate information about the fitness of recurrent neurons. The sup-population architecture of ESP allows a neuron to have some sense of what neurons will be connected to it in a trial. A neuron is guaranteed that a recurrent connection weight r_i will always be associated with neurons from S_i. As the sub-populations specialize, neurons know, with increasing certainty, the kinds of neurons to which they will be connected. Therefore, the recurrent connections to those neurons can be adapted in a meaningful way.

As evolution progresses, each sub-population will decline in diversity. This is a problem, especially in incremental evolution, because a converged population cannot easily adapt to a new task. It can also be a problem within each evaluation-task, because the sub-populations may prematurely converge before the evaluation-task has been optimized. To accomplish task transfer despite convergence, ESP is combined with an iterative search technique known as Delta-Coding.

4.2 Delta-Coding

The idea of Delta-Coding (Whitley et al. 1991) is to search for optimal modifications of the current best solution. In a conventional single-population GA, when the population of candidate solutions has converged, Delta-Coding is invoked by first saving the best solution and then initializing a population of new individuals called Δ-*chromosomes*. The Δ-chromosomes have the same length (number of genes) as the best solution

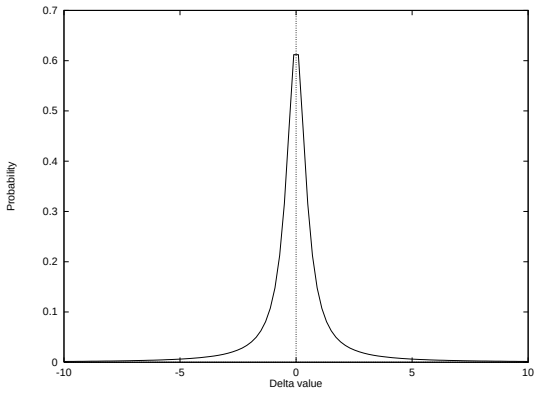


Figure 3: The *Cauchy distribution* for $\alpha = 0.5$. Most of the Δ-values represent small modifications to the best solution, but large values are also possible.

and they consist of values (Δ-*values*) that represent differences from the best solution. The new population is evolved by selecting Δ-chromosomes, adding their Δ-values to the best solution, and evaluating the result. Those Δ-chromosomes that improve the solution are selected for reproduction. Therefore, Delta-Coding explores the hyper-space in a "neighborhood" around the best previous solution. Delta-Coding can be applied multiple times, with successive Δ-populations representing differences to the previous best solution.

ESP implements Delta-Coding with a sub-population architecture. Once the neuron sub-populations have reached minimal diversity, the best solution (i.e. the best network specification) is saved. New sub-populations are then initialized with Δ-chromosomes so that each neuron in the best solution has a dedicated sub-population of Δ-chromosomes that will be evolved to improve it specifically. ESP selects a Δ-chromosome from each sub-population and adds the Δ-values to the connection weights of the neurons in the best-solution. When these sub-populations converge the best Δ-chromosomes are added to the best solution to form the new best solution for the next iteration of the Delta phase.

Delta-Coding was developed to enhance the fine local tuning capability of Genetic Algorithms for numerical optimization. However, its potential for adaptive behavior lies in the facilitation of task transfer. Delta-Coding provides a mechanism for transitioning the evolution into each progressively more demanding task:

$$t_1 \xrightarrow{\Delta} t_2 \xrightarrow{\Delta} t_3 \xrightarrow{\Delta} \cdots \xrightarrow{\Delta} t_n \tag{3}$$

Each $t_i \xrightarrow{\Delta} t_{i+1}$ (Delta transition) involves saving the best network $best(t_i)$ from the last generation of the evaluation task t_i, and initializing sub-populations of Δ-chromosomes that are then evolved to adapt $best(t_i)$ to the next task t_{i+1}. When t_{i+1} has been evolved, $best(t_{i+1})$ (the best modification to $best(t_i)$) is formed by adding the best Δ-chromosomes to $best(t_i)$. The solution $best(t_{i+1})$ will then be adapted to t_{i+2}, and so on.

Since ESP evolves neural networks across tasks, some weights in the best solution may need to change radically. This is especially true if the new task introduces novel input patterns. Therefore, Δ-values need to be generated using a probability density function that concentrates most of the values in the local search space while occasionally permitting values of much larger magnitude. One distribution that has these desirable properties is the *Cauchy* distribution (figure 3):

$$f(x) = \frac{\alpha}{\pi(\alpha^2 + x^2)} \tag{4}$$

With this distribution 50% of the Δ-values will fall within the interval $\pm\alpha$ and 99.9% within the interval $318.3 \pm \alpha$.

5 Experiments

The incremental evolution approach based on ESP and Delta-Coding was tested in the Enemy Avoidance task. Simulations were run both with direct evolution and with incremental evolution with different numbers of enemies and enemy speeds, and the capabilities of the evolved solutions were compared.

5.1 Simulation Setup

For all simulations the following parameter settings were used:

`Sup-population size`← 20
`Sensor range`← 5
`Mutation probability`← 0.1
`Initial random weight range`← [-6.0,6.0]
`Hidden Units`← 8
α (for Δ-values)← 0.5

Evaluation-tasks will be referred to by the notation E_n^s, where n is the number of enemies in the environment and s is their speed. Agents are evolved both directly and incrementally to accomplish $E_2^{0.3}$ (i.e. two enemies with a speed of 0.3). The incremental evolution is then pushed further by increasing the enemy speed to 0.5 ($E_2^{0.5}$). This is done to illustrate how ESP can transfer over multiple tasks with the aid of Delta-Coding. An Agent is considered to have accomplished the task if it can avoid the enemies for more than 1000 time steps. At each generation, 200 networks are constructed and evaluated in the following way: An Agent is placed at a random position in the grid-world, and the enemies are placed at random positions 4 grid-cells from the Agent. The Agent and each enemy take an action each time-step until the Agent occupies the same grid-cell as an enemy, meaning the Agent has been caught. For every time-step in which the enemies are avoided (i.e. the trial-length, or "lifetime"), the Agent is awarded a point. A fitness score of 100 means that the Agent avoided the enemies for 100 time-steps. By placing the Agent and enemies near each other in a relatively small environment the density of task-relevant experiences per trial is maximized. This setup promotes the appropriate general behavior and limits the extent to which the Agent is imprinted with environment-specific characteristics such a grid-world size and enemy speed. Trial lengths are not fixed and are determined by individual performance.

5.2 Direct Evolution

For the Direct evolution simulations, the evaluation-task remains constant throughout evolution. The networks are subjected to $E_2^{0.3}$ in a 13×13 grid-world from the beginning of their evolution. The lower plot in figure 4 shows the results for direct evolution.

As can be clearly seen in this figure, direct evolution of is not powerful enough to solve $E_2^{0.3}$. All of the networks in the first generation perform too poorly to provide adequate differentiation for reproduction; the environment is simply too difficult for any single individual to perform significantly above average. The networks improve slightly over the first 20 generations but, invariably, become trapped in a region of the weight space where the sub-populations have converged before basic task skills have been acquired. These individuals will at best move around the environment a few times in a mechanical fashion before getting trapped against a wall.

5.3 Incremental Evolution

For incremental evolution the population is first evolved on the task $E_1^{0.3}$ in a 10×10 grid-world. Once this initial task has been adequately evolved, the best-performing network is saved and Delta-Coding is invoked to evolve $E_2^{0.3}$. At the same time, the grid-world is enlarged from 10×10 to 13×13 in order to provide additional room for the Agent to maneuver in the presence of a second enemy. After $E_2^{0.3}$, evolution transitions through $E_2^{0.4}$ into $E_2^{0.5}$ by increasing the enemies' speed by 0.2 in two steps. The evolution schedule is thus:

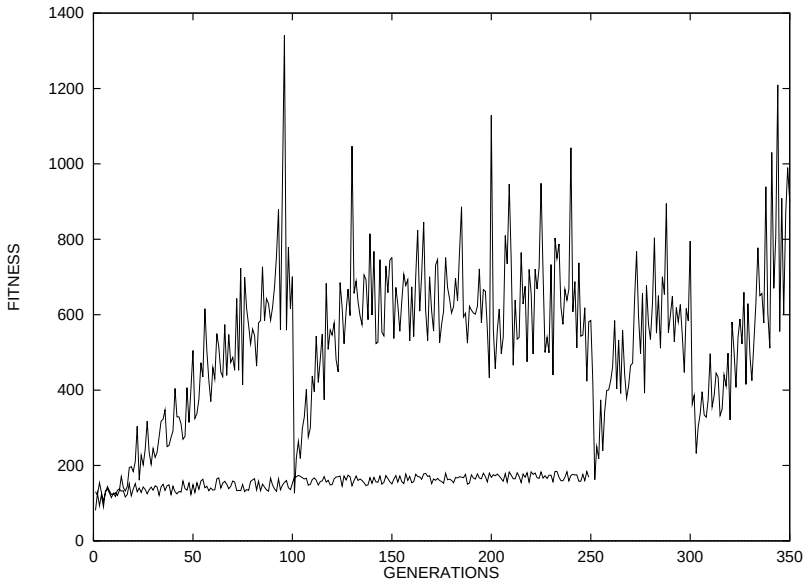


Figure 4: Fitness of the *best* network from each generation for both direct and incremental evolution. The lower curve is the best fitness of the networks formed each generation during the direct evolution of $E_2^{0.3}$. The upper is the best fitness for the incremental approach. The abrupt drops in this curve indicate the task transitions. The transition from $E_1^{0.3}$ to $E_2^{0.3}$ occurs at 100 generations, from $E_2^{0.3}$ to $E_2^{0.4}$ at 250, from $E_2^{0.4}$ to $E_2^{0.5}$ at 300.

$$E_1^{0.3} \xrightarrow{\Delta} E_2^{0.3} \xrightarrow{\Delta} E_2^{0.4} \xrightarrow{\Delta} E_2^{0.5}$$

Each phase of this evolutionary regime will be discussed separately in the following paragraphs. Figure 4 summarizes the results, showing the maximum fitness per generation. The abrupt drop-offs indicate the points where a new task is introduced.

5.3.1 Avoiding One Enemy ($E_1^{0.3}$)

This initial stage of incremental evolution serves to bootstrap the entire incremental process by presenting a task that can be evolved from an initial random population. When $E_1^{0.3}$ is used as the initial evaluation-task, there is sufficient variation in the performance of networks to direct the genetic search. In this easier environment, some networks are able to survive significantly longer than others. Importantly, they survive longer by displaying a primitive form of the fundamental task skills required for Enemy Avoidance. In the early stages of evolution, a useful and formative skill is that of simply being able to keep moving. Those individuals that can keep moving and do not get stuck against the wall then have a chance to discover that moving away from an enemy is beneficial. Some Agent may do well avoiding enemies from the east, another from the west, while another from the north or south. Over the course of evolution the genetic recombination of these skills eventually produces a well-adapted individual that can avoid the enemy from all sides. By favoring these individuals, subsequent generations will inherit and improve upon those skills. The longer as Agent survives, the more clearly it demonstrates competence in the task by enduring a greater exposure to the environment's dynamics. By the end of this phase individuals can avoid the enemy for more than 1000 moves. At this point individuals have evolved to avoid the enemy not only by moving away from it but also by staying away from the corners.

5.3.2 Avoiding Two Enemies ($E_1^{0.3} \stackrel{\Delta}{\rightarrow} E_2^{0.3}$)

Although contending with two enemies is decidedly more difficult than the single enemy scenario, much of the behavior learned in $E_1^{0.3}$ can be applied successfully to $E_2^{0.5}$. Tactically, the two enemies can often be regarded as a single entity. For instance, if both enemies come to occupy the same sensory array sector, the appropriate action for the Agent in this situation would be the same if there were actually only one enemy. Since the enemies tend to be near each other when they are near the Agent, many situations arise where the policy learned in $E_1^{0.3}$ can be exploited. In addition, an $E_2^{0.3}$ trial will contain many periods where the Agent must only avoid a single enemy. Scenarios that pose tactical difficulty are those in which the enemies approach the Agent from opposite sides. Here, the behavior learned in $E_1^{0.3}$ may interfere with the behavior needed for $E_2^{0.3}$ and must be adapted to accommodate a second enemy. From a more strategic perspective, the presence of a second enemy renders the environment qualitatively more difficult. The Agent needs to be able to predict the possible actions to two adversaries that move independently. While it is maneuvering to avoid an enemy that is within sensor range, it must maintain some idea of where the other enemy is located if it is going to be able to avoid being trapped.

Incremental evolution creates individuals that are responsive and resilient with respect to enemy behavior. There is no coarse, patterned strategy in evidence. The Agent behaves with considerable parsimony, taking those actions that allow it to avoid the enemies in the immediate state and guarantee some minimum of safety in subsequent states. It does not simply flee from the enemies into a remote part of the environment where it will fall into inescapable peril, as in direct evolution.

5.3.3 Increasing Enemy Speed ($E_2^{0.3} \stackrel{\Delta}{\rightarrow} E_2^{0.4} \stackrel{\Delta}{\rightarrow} E_2^{0.5}$)

This stage is included to emphasize how ESP and Delta-Coding can effectively make multiple transfers. The transition from an enemy speed of 0.3 to 0.5 is especially challenging in the presence of multiple enemies. This final speed means that both enemies are moving at an average of one grid-cell for every two time-steps, which means that on average, there is an enemy action taken every time-step. To make this transition more rapid it is performed in two stages: $E_2^{0.3} \stackrel{\Delta}{\rightarrow} E_2^{0.4}$ and $E_2^{0.4} \stackrel{\Delta}{\rightarrow} E_2^{0.5}$. Within 50 generations networks are already performing as well as they did on the previous task.

As enemy speed is increased, an Agent needs to better anticipate the possible consequences of its own actions. By the time the entire incremental process is completed an Agent can elude two relatively fast moving enemies for over 1000 time-steps in a small (13×13) grid-world. In order to do so, an Agent has to exploit the entire spatial extent of the environment as much as possible, and try to keep the enemies clustered together. The Agent will move around the enemies coming close to both of them so that they are more likely come closer to each other and become more manageable. It is important note that the behavior that has been acquired through this incremental process is general, and also retains the ability to perform equally well on the earlier tasks $E_1^{0.3}$, $E_2^{0.3}$, and $E_2^{0.4}$

6 Discussion and Future Work

The Enemy Avoidance experiments illustrate the potential of incremental genetic search. This method is applicable to any task that can be naturally decomposed into a sequence of increasingly complex tasks. In Artificial Life and robot control tasks such layers of behavior can usually be easily found subsumed by the goal-task (Brooks 1986). For example, avoiding moving obstacles while following a wall could be evolved incrementally. The robot could first be evolved to move around avoiding stationary obstacles, then obstacles moving at slow speeds. Later, the speed of the obstacles would be increased, and the robot would be evolved to follow a wall.

Once evolved, the controller can always serve as a starting point from which to build increasingly sophisticated behavior through continued application of ESP/Delta-Coding. In fact, the approach can be used to modify any neural network regardless of how it was trained. A major direction of future work is to apply ESP/Delta-Coding to challenging tasks in the real world, such as game playing and robot control. Extensions

of the algorithm will also be tested, such as incrementally adding hidden units to the network by evolving new sub-populations.

7 Conclusion

This paper shows that even when a task is too difficult to evolve directly, Neuro-Evolution can be applied incrementally to achieve the desired complex behavior. In this approach, Enforced Sub-Populations allows evolution of recurrent networks, which are necessary for tasks that require memory. The Delta-Coding technique allows evolution to transition between tasks even when the population has lost diversity during the previous task. The approach should be applicable to many real world domains such as game playing and robot control, as well as Artificial Life , where often a natural hierarchy of behaviors from simple to complex exists.

Appendix A: The algorithm for choosing enemy actions.

Each enemy in the dynamic environment behaves randomly. On each step, $(1-v)\%$ of the time (where $v \in [0,1]$ is user-defined) the enemy will not move, and $v\%$ of the time it will choose one of the four actions, A_0 (north), A_1 (south), A_2 (east), and A_3 (north), according to the following distribution:

$$prob(A_i) = P_i/(P_0 + P_1 + P_2 + P_3),$$

where

$$P_i = \begin{cases} 0 & \text{if } A_i \text{ will result in collision with obstacles,} \\ exp(0.33 \cdot W(angle) \cdot T(dist)) & \text{otherwise,} \end{cases}$$

$$angle = \text{angle between the direction of action } A_i \text{ and the direction from the enemy to the agent,}$$

$$dist = \text{distance between the enemy and the agent,}$$

$$W(angle) = (180 - |angle|)/180,$$

$$T(dist) = \begin{cases} 15 - dist & \text{if } dist \leq 4, \\ 9 - dist/2 & \text{if } dist \leq 15, \\ 1 & \text{otherwise.} \end{cases}$$

References

Brooks, R. A. (1986). A robust layered control system for a mobile robot. *IEEE Journal of Robotics and Automation*, 2(10).

Cliff, D., Harvey, I., and Husbands, P. (1992). Incremental evolution of neural network architectures for adaptive behavior. Technical Report CSRP256, School of Cognitive and Computing Sciences, University of Sussex, Brighton, UK.

Colombetti, M., and Dorigo, M. (1992). Learning to control an autonomous robot by distributed genetic algorithms. In Meyer, J. A., Roitblat, H. L., and Wilson, S. W., editors, *From Animals to Animats 2, Proceedings of the 2nd International Conference on Simulation of Adaptive Behavior*. MIT Press.

Elman, J. L. (1991). Incremental learning, or The importance of starting small. In *Proceedings of the 13th Annual Conference of the Cognitive Science Society*, 443–448. Hillsdale, NJ: Erlbaum.

Law, D., and Miikkulainen, R. (1994). Grounding robotic control with genetic neural networks. Technical Report AI93-223, Department of Computer Sciences, The University of Texas at Austin.

Lin, L.-J. (1992). Self-improving reactive agents based on reinforcement learning, planning, and teaching. *Machine Learning*, 8:293–321.

Miller, G., and Cliff, D. (1994). Co-evolution of pursuit and evasion i: Biological and game-theoretic foundations. Technical Report CSRP311, School of Cognitive and Computing Sciences, University of Sussex.

Moriarty, D. E., and Miikkulainen, R. (1995). Efficient learning from delayed rewards through symbiotic evolution. In Prieditis, A., and Russell, S., editors, *Machine Learning: Proceedings of the 12th Annual Conference*.

Moriarty, D. E., and Miikkulainen, R. (1996a). Evolving obstacle avoidance behavior in a robot arm. In *From Animals to Animats: The Fourth International Conference on Simulation of Adaptive Behavior (SAB96)*.

Moriarty, D. E., and Miikkulainen, R. (1996b). Hierarchical evolution of neural networks. Technical Report AI96-242, Department of Computer Science, The University of Texas at Austin.

Nolfi, S., Elman, J. L., and Parisi, D. (1990). Learning and evolution in neural networks. Technical Report 9019, Center for Research in Language, University of California, San Diego.

Nolfi, S., and Parisi, D. (1994). Good teaching inputs do not correspond to desired responses in ecological neural networks. *Neural Processing Letters*, 1(2):1–4.

Nolfi, S., and Parisi, D. (1995). Learning to adapt to changing environments in evolving neural networks. Technical Report 95-15, Institute of Psychology, National Research Council, Rome, Italy.

Pratt, L. (1993). *Transferring Previously Learned Back-Propagation Neural Networks to New Learning Tasks.* PhD thesis, Rutgers University, Department of Computer Science.

Pratt, L. (1994). Experiments on the transfer of knowledge between neural networks. In Hanson, S., Drastal, G., and Rivest, R., editors, *Computational Learning Theory and Natural Learning Systems, Constraints and Prospects*, 523–560. MIT Press.

Ring, M. B. (1994). *Continual Learning in Reinforcement Environments.* PhD thesis, Department of Computer Sciences, University of Texas at Austin, Austin, Texas 78712.

Thrun, S. (1996). *Explanation-Based Neural Network Learning: A Lifelong Learning Approach.* Kluwer Academic Publishers.

Whitley, D., Mathias, K., and Fitzhorn, P. (1991). Delta-coding: An iterative search strategy for genetic algortihms. In *Proceedings of the Fourth International Conference on Genetic Algorithms.* Los Altos, CA: Morgan Kaufmann.

Wilson, S. W. (1991). The animat path to AI. In Meyer, J., and Wilson, S., editors, *Proceedings of the First International Conference on Simulation of Adaptive Behavior*, 15–21. MIT Press/Bradford Books.

Yamauchi, B., and Beer, R. (1994). Integrating reactive, sequential, and learning behavior using dynamical neural networks. In Cliff, D., Husbands, P., Meyer, J. A., and Wilson, S., editors, *From Animals to Animats 3, Proceedings of the 3rd International Conference on Simulation of Adaptive Behavior.* MIT Press/Bradford Books.